

Backtesting avec Vectorbt / Vectorbt Pro

Table of Contents

Backtesting de stratégies de trading avec Vectorbt / Vectorbt Pro	1
Paramètres communs à tous les exemples.....	2
1. Présentation de Vectorbt.....	2
2. Installation et mise en place	3
3. Concepts fondamentaux : moteur vectorisé.....	3
Exemple 1 — Buy & Hold	4
Exemple 2 — Croisement de moyennes mobiles (SMA 50/200)	5
Exemple 3 — Retour à la moyenne (RSI 14).....	6
Exemple 4 — Bandes de Bollinger (20/2).....	6
Exemple 5 — MACD (12/26/9)	7
Exemple 6 — Portefeuille multi-actifs	8
Exemple 7 — Dimensionnement de position et risque	9
Exemple 8 — Stop-loss et take-profit.....	9
Exemple 9 — Analyse des métriques de performance	10
Exemple 10 — Optimisation de paramètres.....	11
Exemple 11 — Validation walk-forward / hors-échantillon	12
Tableau comparatif : LEAN vs Vectorbt.....	13
Exemple 12 — Stratégie avancée complète	14
15. Conclusion.....	16

Backtesting de stratégies de trading avec Vectorbt / Vectorbt Pro

Document formatif — Bibliothèque **Vectorbt** (open-source) et **Vectorbt Pro** (version étendue), approche **vectorisée** en Python.

Ce document est le jumeau de celui consacré à **QuantConnect LEAN** : il contient **exactement les mêmes exemples**, mais transposés aux idiomes vectorisés de Vectorbt.

Paramètres communs à tous les exemples

Pour que les exemples soient strictement comparables d'une plateforme à l'autre, ils partagent tous la même configuration :

Paramètre	Valeur
Capital initial	100 000 \$
Frais de transaction	0,1 % (0,001)
Période de backtest	2020-01-01 → 2023-12-31
Actif principal	SPY
Panier multi-actifs	AAPL, MSFT, GOOG
Résolution	Journalière (1D)
SMA croisement	rapide 50 / lente 200
RSI	période 14, seuils 30 / 70
Bandes de Bollinger	période 20, écart-type 2
MACD	12 / 26 / 9

1. Présentation de Vectorbt

Vectorbt est une bibliothèque Python de backtesting et d'analyse quantitative bâtie sur **pandas**, **NumPy** et **Numba** (compilation JIT). Sa philosophie est radicalement différente de celle d'un moteur événementiel : au lieu de simuler le marché barre par barre, Vectorbt **calcule des signaux sur toute la série temporelle d'un coup**, sous forme de tableaux booléens, puis simule le portefeuille de manière vectorisée. Résultat : on peut tester **des milliers de variantes** de paramètres en quelques secondes.

Deux versions coexistent :

- **Vectorbt (open-source)** — gratuite, importée via `import vectorbt as vbt`. Classes d'indicateurs `vbt.MA`, `vbt.RSI`, `vbt.BBANDS`, `vbt.MACD`.
- **Vectorbt Pro (vbtpro)** — version commerciale plus rapide et plus riche : alias `vbt.PF` pour Portfolio, accès `data.run("talib:...")`, classe `vbt.Splitter` pour le walk-forward, méthode `vbt.YFData.pull(...)` au lieu de `download(...)`.

Dans ce document, le code est écrit pour **Vectorbt open-source** ; chaque exemple signale en encadré la variante **Pro** lorsqu'elle diffère.

2. Installation et mise en place

```
# Vectorbt open-source
pip install -U vectorbt

# Avec toutes les dépendances optionnelles (graphiques interactifs, etc.)
pip install -U "vectorbt[full]"

# Vectorbt Pro (nécessite une licence et l'accès au dépôt privé)
pip install -U "vectorbtpro[full]"
```

Imports standards utilisés dans tout le document :

```
import vectorbt as vbt          # open-source
# import vectorbtpro as vbt     # variante Pro : même alias vbt
import numpy as np
import pandas as pd
```

On récupère les prix de clôture journaliers (source Yahoo Finance intégrée) :

```
# Open-source
prix = vbt.YFData.download(
    "SPY", start="2020-01-01", end="2023-12-31"
).get("Close")

# Pro : utiliser .pull() au lieu de .download()
# prix = vbt.YFData.pull("SPY", start="2020-01-01", end="2023-12-31").get("Close")
```

3. Concepts fondamentaux : moteur vectorisé

La logique Vectorbt tient en trois temps :

1. **Calculer des indicateurs** sur toute la série (pas de boucle).
2. **Produire des signaux** entries et exits : deux tableaux booléens alignés sur les prix.
3. **Simuler** le portefeuille avec `Portfolio.from_signals(...)`, puis lire les statistiques.

```
import vectorbt as vbt

# 1) Données : prix de clôture journaliers de SPY
prix = vbt.YFData.download("SPY", start="2020-01-01", end="2023-12-31").get("Close")
```

```
# 2) Signaux booléens (ici, signaux factices pour illustrer la structure)
entries = prix > prix.shift(1)  # True quand le prix monte
exits = prix < prix.shift(1)    # True quand le prix baisse

# 3) Simulation du portefeuille (capital 100 000 $, frais 0,1 %)
pf = vbt.Portfolio.from_signals(
    prix, entries, exits,
    init_cash=100_000, fees=0.001, freq="1D",
)
print(pf.total_return())
```

Idées-clés :

- À aucun moment on n'écrit de boucle « pour chaque jour » : tout est calculé sur les vecteurs entiers. C'est la source de la rapidité.
- `entries` et `exits` ont **la même forme** que `prix`. Si `prix` est un `DataFrame` à plusieurs colonnes, toute la chaîne traite les colonnes simultanément.
- **Attention au look-ahead bias** : c'est à vous de garantir qu'un signal calculé à l'instant t n'utilise pas d'information de $t+1$. Les indicateurs Vectorbt sont causaux par défaut, mais une manipulation maladroite (`shift` dans le mauvais sens) peut introduire un biais.
- `freq="1D"` indique la fréquence des barres : indispensable pour annualiser correctement le Sharpe.

Exemple 1 — Buy & Hold

Objectif pédagogique : la stratégie de référence. On achète SPY au premier jour et on conserve jusqu'à la fin. Toute stratégie active devra battre ce *benchmark*.

```
import vectorbt as vbt

prix = vbt.YFData.download("SPY", start="2020-01-01", end="2023-12-31").get("Close")

# from_holding : achète tout au départ et conserve.
pf = vbt.Portfolio.from_holding(
    prix,
    init_cash=100_000,
    fees=0.001,
    freq="1D",
)
```

```
print(f"Rendement total : {pf.total_return():.2%}")
```

À retenir : `Portfolio.from_holding` est le raccourci dédié au Buy & Hold. `Vectorbt` investit la totalité du capital à la première barre et calcule ensuite l'ensemble des métriques accessibles via `pf.stats()`.

Variante Pro : `pf = vbt.PF.from_holding(prix, init_cash=100_000, fees=0.001, freq="1D")`.

Exemple 2 — Croisement de moyennes mobiles (SMA 50/200)

Objectif pédagogique : la stratégie de suivi de tendance la plus classique (*golden cross / death cross*). On est long quand la SMA 50 passe au-dessus de la SMA 200, on liquide au croisement inverse.

```
import vectorbt as vbt

prix = vbt.YFData.download("SPY", start="2020-01-01", end="2023-12-31").get("Close")

# Deux moyennes mobiles simples calculées sur toute la série.
fast_ma = vbt.MA.run(prix, 50)
slow_ma = vbt.MA.run(prix, 200)

# Signaux : True exactement au moment du croisement.
entries = fast_ma.ma_crossed_above(slow_ma)    # croisement haussier
exits = fast_ma.ma_crossed_below(slow_ma)      # croisement baissier

pf = vbt.Portfolio.from_signals(
    prix, entries, exits,
    init_cash=100_000, fees=0.001, freq="1D",
)
print(f"Rendement total : {pf.total_return():.2%}")
```

À retenir : `vbt.MA.run(prix, période)` calcule la moyenne mobile sur tout l'historique. Les méthodes `ma_crossed_above` / `ma_crossed_below` renvoient des tableaux booléens valant `True` uniquement à la barre du croisement — exactement la logique d'entrée/sortie de l'Exemple 2 LEAN.

Variante Pro : `fast_sma = data.run("talib_func:sma", timeperiod=50)` puis `entries = fast_sma.vbt.crossed_above(slow_sma)`.

Exemple 3 — Retour à la moyenne (RSI 14)

Objectif pédagogique : stratégie contrarienne. On achète en survente ($RSI < 30$) et on vend en surachat ($RSI > 70$).

```
import vectorbt as vbt

prix = vbt.YFData.download("SPY", start="2020-01-01", end="2023-12-31").get("Close")

# RSI sur 14 périodes.
rsi = vbt.RSI.run(prix, window=14)

# Signaux : achat sous 30, vente au-dessus de 70.
entries = rsi.rsi_below(30)
exits = rsi.rsi_above(70)

pf = vbt.Portfolio.from_signals(
    prix, entries, exits,
    init_cash=100_000, fees=0.001, freq="1D",
)
print(f"Rendement total : {pf.total_return():.2%}")
```

À retenir : `vbt.RSI.run` expose des comparateurs prêts à l'emploi `rsi_below` / `rsi_above` qui génèrent directement les signaux booléens. La logique « acheter bas / vendre haut » est l'inverse du suivi de tendance, exactement comme dans l'Exemple 3 LEAN.

Variante Pro : `rsi = data.run("talib:RSI", timeperiod=14)`, puis comparer le tableau de valeurs avec `rsi.real.vbt < 30`.

Exemple 4 — Bandes de Bollinger (20/2)

Objectif pédagogique : retour à la moyenne sur volatilité. On achète quand le prix touche la bande inférieure, on vend quand il revient sur la bande médiane (ou supérieure).

```
import vectorbt as vbt

prix = vbt.YFData.download("SPY", start="2020-01-01", end="2023-12-31").get("Close")

# Bandes de Bollinger : période 20, 2 écarts-types.
bb = vbt.BBANDS.run(prix, window=20, alpha=2)

# Entrée : Le prix franchit la bande inférieure (survente).
entries = prix < bb.lower
```

Sortie : Le prix repasse au-dessus de la bande médiane.

```
exits = prix > bb.middle
```

```
pf = vbt.Portfolio.from_signals(  
    prix, entries, exits,  
    init_cash=100_000, fees=0.001, freq="1D",  
)  
print(f"Rendement total : {pf.total_return():.2%}")
```

À retenir : `vbt.BBANDS.run` expose les trois bandes via `bb.lower`, `bb.middle`, `bb.upper`. La comparaison `prix < bb.lower` produit directement un masque booléen sur toute la série — l'équivalent vectorisé du test prix/bande de l'Exemple 4 LEAN.

Variante Pro : `bb = data.run("talib:BBANDS", timeperiod=20, nbdevup=2, nbdevdn=2).`

Exemple 5 — MACD (12/26/9)

Objectif pédagogique : suivi de tendance via le MACD. On est long quand la ligne MACD passe au-dessus de sa ligne de signal.

```
import vectorbt as vbt  
  
prix = vbt.YFData.download("SPY", start="2020-01-01", end="2023-12-31").get("Close")  
  
# MACD : EMA rapide 12, EMA lente 26, signal 9.  
macd = vbt.MACD.run(prix, fast_window=12, slow_window=26, signal_window=9)  
  
# Signaux : croisement de la ligne MACD et de sa ligne de signal.  
entries = macd.macd_above(macd.signal) # MACD au-dessus du signal  
exits = macd.macd_below(macd.signal) # MACD sous le signal  
  
pf = vbt.Portfolio.from_signals(  
    prix, entries, exits,  
    init_cash=100_000, fees=0.001, freq="1D",  
)  
print(f"Rendement total : {pf.total_return():.2%}")
```

À retenir : `vbt.MACD.run` expose `macd.macd` (la ligne MACD) et `macd.signal` (la ligne de signal). Les helpers `macd_above` / `macd_below` traduisent le croisement en signaux booléens, exactement comme la comparaison des deux lignes dans l'Exemple 5 LEAN.

Variante Pro : `macd = data.run("talib:MACD", fastperiod=12, slowperiod=26, signalperiod=9).`

Exemple 6 — Portefeuille multi-actifs

Objectif pédagogique : appliquer la même stratégie (croisement SMA 50/200) simultanément à plusieurs titres (AAPL, MSFT, GOOG), avec une allocation égale ($\approx 33\%$ par titre quand le signal est actif).

C'est ici que la vectorisation brille : on passe un DataFrame à **plusieurs colonnes** et toute la chaîne traite les trois titres d'un coup.

```
import vectorbt as vbt

tickers = ["AAPL", "MSFT", "GOOG"]

# prix est un DataFrame : une colonne par titre.
prix = vbt.YFData.download(
    tickers, start="2020-01-01", end="2023-12-31", missing_index="drop"
).get("Close")

fast_ma = vbt.MA.run(prix, 50)
slow_ma = vbt.MA.run(prix, 200)

entries = fast_ma.ma_crossed_above(slow_ma)
exits = fast_ma.ma_crossed_below(slow_ma)

# group_by + cash_sharing : un seul portefeuille partagé entre les 3 titres.
# Allocation égale : ~1/3 du capital par position active.
pf = vbt.Portfolio.from_signals(
    prix, entries, exits,
    init_cash=100_000, fees=0.001, freq="1D",
    size=1/3, size_type="targetpercent",
    group_by=True, cash_sharing=True,
)
print(pf.total_return())
```

À retenir : `size=1/3` avec `size_type="targetpercent"` demande d'allouer un tiers de la valeur du portefeuille à chaque position active — l'équivalent des trois `set_holdings(symbol, 0.33)` de l'Exemple 6 LEAN. `group_by=True` et `cash_sharing=True` mutualisent la trésorerie entre les trois titres.

Variante Pro : remplacer `vbt.Portfolio` par `vbt.PF` ; les arguments sont identiques.

Exemple 7 — Dimensionnement de position et risque

Objectif pédagogique : ne pas investir 100 % à chaque signal. Ici, on n'engage que **95 % du capital** par entrée (en gardant un coussin de liquidités) sur la stratégie de croisement SMA.

```
import vectorbt as vbt

prix = vbt.YFData.download("SPY", start="2020-01-01", end="2023-12-31").get("Close")

fast_ma = vbt.MA.run(prix, 50)
slow_ma = vbt.MA.run(prix, 200)
entries = fast_ma.ma_crossed_above(slow_ma)
exits = fast_ma.ma_crossed_below(slow_ma)

pf = vbt.Portfolio.from_signals(
    prix, entries, exits,
    init_cash=100_000, fees=0.001, freq="1D",
    # On n'engage que 95 % du capital par position.
    size=0.95, size_type="percent",
)
print(f"Rendement total : {pf.total_return():.2%}")
```

À retenir : `size=0.95` avec `size_type="percent"` plafonne l'exposition à 95 % du capital — l'équivalent exact du `set_holdings(symbol, 0.95)` de l'Exemple 7 LEAN. Le dimensionnement de position est un levier de gestion du risque au moins aussi important que les signaux d'entrée/sortie.

Exemple 8 — Stop-loss et take-profit

Objectif pédagogique : ajouter une sortie en perte (stop-loss à **5 %** sous le prix d'entrée) et une prise de bénéfice (take-profit à **10 %** au-dessus), greffées sur la stratégie de croisement SMA.

C'est l'un des points forts de Vectorbt : stop-loss et take-profit s'activent par deux simples arguments.

```
import vectorbt as vbt

prix = vbt.YFData.download("SPY", start="2020-01-01", end="2023-12-31").get("Close")

fast_ma = vbt.MA.run(prix, 50)
slow_ma = vbt.MA.run(prix, 200)
entries = fast_ma.ma_crossed_above(slow_ma)
```

```

exits = fast_ma.ma_crossed_below(slow_ma)

pf = vbt.Portfolio.from_signals(
    prix, entries, exits,
    init_cash=100_000, fees=0.001, freq="1D",
    sl_stop=0.05,    # stop-loss à 5 %
    tp_stop=0.10,    # take-profit à 10 %
)
print(f"Rendement total : {pf.total_return():.2%}")
print(pf.trades.records_readable[["Entry Price", "Exit Price", "PnL"]].head())

```

À retenir : `sl_stop=0.05` et `tp_stop=0.10` sont exprimés en fraction du prix d'entrée. Le moteur surveille, pour chaque position, le franchissement de ces niveaux à chaque barre — exactement la logique manuelle `prix <= entrée*(1-0,05) / prix >= entrée*(1+0,10)` de l'Exemple 8 LEAN, mais en une ligne. On peut aussi ajouter `ts_stop=...` pour un stop suiveur.

Exemple 9 — Analyse des métriques de performance

Objectif pédagogique : ne pas se contenter du rendement total. On veut le ratio de Sharpe, le drawdown maximum, le taux de réussite, etc.

Dans Vectorbt, la méthode `pf.stats()` **calcule automatiquement** toutes ces métriques en une fois. On peut aussi y accéder individuellement.

```

import vectorbt as vbt

prix = vbt.YFData.download("SPY", start="2020-01-01", end="2023-12-31").get("Close")

fast_ma = vbt.MA.run(prix, 50)
slow_ma = vbt.MA.run(prix, 200)
entries = fast_ma.ma_crossed_above(slow_ma)
exits = fast_ma.ma_crossed_below(slow_ma)

pf = vbt.Portfolio.from_signals(
    prix, entries, exits,
    init_cash=100_000, fees=0.001, freq="1D",
)

# Rapport complet (équivalent du tableau de statistiques de LEAN).
print(pf.stats())

# Accès individuel aux métriques clés.

```

```

print(f"Rendement total      : {pf.total_return():.2%}")
print(f"Ratio de Sharpe     : {pf.sharpe_ratio():.2f}")
print(f"Drawdown maximum    : {pf.max_drawdown():.2%}")
print(f"Taux de réussite     : {pf.trades.win_rate():.2%}")
print(f"Valeur finale        : {pf.final_value():.0f} $")

```

À retenir : `pf.stats()` renvoie une série pandas avec rendement total, Sharpe, Sortino, Calmar, drawdown maximum, nombre de trades, win rate, profit factor, etc. C'est l'équivalent direct du calcul manuel fait dans `on_end_of_algorithm` à l'Exemple 9 LEAN — sauf qu'ici tout est fourni d'office. Comparez toujours votre Sharpe à celui d'un simple Buy & Hold.

Exemple 10 — Optimisation de paramètres

Objectif pédagogique : trouver le meilleur couple (SMA rapide, SMA lente) parmi une grille — rapide $\in \{10, 20, 50\}$, lente $\in \{100, 150, 200\}$.

C'est le cas d'usage emblématique de `Vectorbt` : on passe des **listes** de fenêtres et toutes les combinaisons sont testées **en une seule passe vectorisée**, chacune dans sa propre colonne.

```

import vectorbt as vbt

prix = vbt.YFData.download("SPY", start="2020-01-01", end="2023-12-31").get("Close")

# Grille de paramètres.
fast_windows = [10, 20, 50]
slow_windows = [100, 150, 200]

# vbt génère toutes les combinaisons rapide x lente d'un coup.
fast_ma = vbt.MA.run(prix, fast_windows, short_name="fast", per_column=False)
slow_ma = vbt.MA.run(prix, slow_windows, short_name="slow", per_column=False)

entries = fast_ma.ma_crossed_above(slow_ma)
exits = fast_ma.ma_crossed_below(slow_ma)

pf = vbt.Portfolio.from_signals(
    prix, entries, exits,
    init_cash=100_000, fees=0.001, freq="1D",
)

# Sharpe par combinaison, puis meilleur couple.
sharpe = pf.sharpe_ratio()

```

```
print(sharpe)
print("Meilleure combinaison :", sharpe.idxmax(), "| Sharpe =", round(sharpe.max(), 2))
```

À retenir : là où LEAN doit relancer un backtest complet par combinaison via son optimiseur (Exemple 10 LEAN), Vectorbt évalue **les 9 combinaisons simultanément** dans un seul Portfolio. `pf.sharpe_ratio()` renvoie alors une série indexée par `(fast_window, slow_window)`, et `.idxmax()` donne le gagnant. **Attention au sur-optimisation (overfitting)** : une grille trop fine donne des paramètres « parfaits » sur le passé mais fragiles en réel.

Variante Pro : `vbt.MA.run_combs(prix, window=np.arange(10, 201), r=2, short_names=["fast", "slow"])` pour explorer des milliers de couples d'un coup.

Exemple 11 — Validation walk-forward / hors-échantillon

Objectif pédagogique : valider la robustesse en séparant la période d'**entraînement** (in-sample, où l'on optimise) de la période de **test** (out-of-sample, jamais vue). On optimise sur 2020-2022 et on valide sur 2023.

```
import vectorbt as vbt

prix = vbt.YFData.download("SPY", start="2020-01-01", end="2023-12-31").get("Close")

# 1) Découpage in-sample / out-of-sample.
is_prix = prix.loc["2020-01-01":"2022-12-31"]    # entraînement
oos_prix = prix.loc["2023-01-01":"2023-12-31"]    # validation

fast_windows = [10, 20, 50]
slow_windows = [100, 150, 200]

def backtest(p, fast, slow):
    fma = vbt.MA.run(p, fast, short_name="fast", per_column=False)
    sma = vbt.MA.run(p, slow, short_name="slow", per_column=False)
    entries = fma.ma_crossed_above(sma)
    exits = fma.ma_crossed_below(sma)
    return vbt.Portfolio.from_signals(
        p, entries, exits, init_cash=100_000, fees=0.001, freq="1D"
    )

# 2) Optimisation SUR L'IN-SAMPLE uniquement.
pf_is = backtest(is_prix, fast_windows, slow_windows)
meilleur = pf_is.sharpe_ratio().idxmax()    # ex. (50, 200)
print("Meilleur couple in-sample :", meilleur)
```

```
# 3) Validation du seul couple retenu SUR L'OUT-OF-SAMPLE.
best_fast, best_slow = meilleur
pf_oos = backtest(oos_prix, best_fast, best_slow)
print(f"Sharpe in-sample : {pf_is.sharpe_ratio().max():.2f}")
print(f"Sharpe out-of-sample : {pf_oos.sharpe_ratio():.2f}")
```

Protocole :

1. On optimise la grille **uniquement** sur 2020-2022 (in-sample).
2. On retient le meilleur couple (fast, slow).
3. On applique ce seul jeu de paramètres sur 2023 (out-of-sample).
4. Si le Sharpe s'effondre hors-échantillon, la stratégie est sur-optimisée.

À retenir : le découpage manuel in-sample / out-of-sample est l'équivalent exact des phases train / test paramétrées de l'Exemple 11 LEAN. La discipline in-sample / out-of-sample est la protection principale contre l'illusion de performance.

Variante Pro : `vbt.Splitter.from_rolling(prix.index, ...)` automatise le walk-forward glissant (plusieurs fenêtres train/test successives) en quelques lignes.

Tableau comparatif : LEAN vs Vectorbt

Les deux plateformes répondent au même besoin — backtester des stratégies en Python — mais selon deux philosophies opposées. Le tableau ci-dessous synthétise leurs points communs et leurs différences.

Similitudes

Point commun	Détail
Objectif	Backtester des stratégies de trading sur données historiques
Langage	Python (LEAN propose aussi C#)
Indicateurs intégrés	SMA, EMA, RSI, Bandes de Bollinger, MACD, ATR, etc.
Logique entrée/sortie	Signaux d'achat et de vente, gestion de positions
Gestion du risque	Stop-loss, take-profit, dimensionnement de position
Frais	Modélisation des coûts de transaction
Métriques	Rendement, Sharpe, drawdown, taux de réussite, etc.
Optimisation	Recherche de paramètres et validation hors-échantillon
Licence	Code source ouvert (LEAN entièrement ; Vectorbt en open-source, Pro commercial)

Différences

Aspect	QuantConnect LEAN	Vectorbt / Vectorbt Pro
Paradigme	Événementiel (barre par barre)	Vectorisé (toute la série d'un coup)
Langages	Python et C#	Python uniquement
Vitesse de backtest	Plus lente (simulation séquentielle)	Très rapide (NumPy + Numba)
Exploration de paramètres	Un backtest relancé par combinaison	Des milliers de combinaisons en une passe
Réalisme de la microstructure	Élevé (slippage, marge, fills, dividendes, splits)	Simplifié
Look-ahead bias	Impossible par construction	Possible si le code est mal écrit
Passage au trading réel	Natif : le même code passe en live	Surtout recherche/backtest ; live limité
Données	Marketplace intégrée + connecteurs courtiers	Sources externes (yfinance, CSV, API...)
Contrôle fin par ordre	Granulaire et natif	Via callbacks (surtout en Pro)
Courbe d'apprentissage	Cadre structuré QCAgorithm	API pandas/NumPy familière
Coût	Cloud freemium ; moteur gratuit en local	Vectorbt gratuit ; Pro payant

Avantages et inconvénients

	QuantConnect LEAN	Vectorbt / Vectorbt Pro
Avantages	Réalisme élevé ; pas de look-ahead ; passage au live sans réécriture ; données et classes d'actifs intégrées ; gestion fine des ordres	Vitesse extrême ; exploration massive de paramètres ; analyse statistique riche (stats()) ; idéal pour la recherche et le machine learning
Inconvénients	Backtests plus lents ; optimisation coûteuse en calcul ; code plus verbeux ; dépendance à Docker / plateforme en local	Risque de look-ahead ; modélisation moins réaliste ; pas de live natif robuste ; logique par-ordre plus complexe ; fonctions avancées payantes (Pro)

En résumé : on choisit **LEAN** pour la fidélité au marché réel et le passage en production, et **Vectorbt** pour la rapidité de recherche et l'exploration de très nombreuses idées. Les deux sont complémentaires : prototyper et filtrer des idées sous Vectorbt, puis valider et déployer la meilleure sous LEAN, est un flux de travail courant.

Exemple 12 — Stratégie avancée complète

Objectif pédagogique : réunir en une seule stratégie la plupart des concepts précédents. Il s'agit d'une **stratégie multi-actifs de suivi de tendance, filtrée par le régime de marché et pilotée par le risque** :

- **Univers :** AAPL, MSFT, GOOG.
- **Filtre de régime :** on ne prend position que si le prix est au-dessus de sa SMA 200 (tendance de fond haussière).

- **Signal d'entrée** : confirmation de momentum — ligne MACD au-dessus de sa ligne de signal **et** RSI(14) > 50.
- **Dimensionnement par le risque** : on risque **1 % du capital** par position ; la taille se déduit de la distance au stop (volatilité mesurée par l'ATR).
- **Stop suiveur** : $2 \times \text{ATR}(14)$ sous le plus haut atteint depuis l'entrée.
- **Take-profit** : $4 \times \text{ATR}$ au-dessus de l'entrée (ratio risque/rendement 2:1).
- **Limite d'exposition** : 3 positions simultanées au maximum.

```
import vectorbt as vbt
import numpy as np

tickers = ["AAPL", "MSFT", "GOOG"]
data = vbt.YFData.download(
    tickers, start="2020-01-01", end="2023-12-31", missing_index="drop"
)
close = data.get("Close")    # DataFrame : une colonne par actif
high = data.get("High")
low = data.get("Low")

# Les indicateurs vbt ajoutent un niveau de paramètre aux colonnes ; on réaligne
# proprement sur les colonnes d'origine (les tickers) pour pouvoir les combiner.
def aligner(df):
    df = df.copy()
    df.columns = close.columns
    return df

# --- Indicateurs calculés sur tout l'historique, pour les 3 actifs à la fois ---
sma200 = aligner(vbt.MA.run(close, 200).ma)
macd = vbt.MACD.run(close, fast_window=12, slow_window=26, signal_window=9)
macd_line = aligner(macd.macd)
macd_signal = aligner(macd.signal)
rsi = aligner(vbt.RSI.run(close, window=14).rsi)
atr = aligner(vbt.ATR.run(high, low, close, window=14).atr)

# --- Filtre de régime + confirmation de momentum ---
regime_haussier = close > sma200
momentum = (macd_line > macd_signal) & (rsi > 50)
entries = regime_haussier & momentum

# Sortie « Logique » : perte du régime ou du momentum.
# Les stops ATR ci-dessous gèrent les sorties de risque.
exits = (close < sma200) | (macd_line < macd_signal)
```

```

# --- Stops dynamiques basés sur l'ATR, exprimés en fraction du prix ---
ts_stop = (2.0 * atr) / close    # stop suiveur à 2 x ATR
tp_stop = (4.0 * atr) / close    # take-profit à 4 x ATR

# --- Dimensionnement par Le risque : 1 % du capital, taille en nombre d'actions ---
# actions = (risque en $) / (distance au stop en $)
risque_dollars = 100_000 * 0.01
taille_actions = risque_dollars / (2.0 * atr)

pf = vbt.Portfolio.from_signals(
    close, entries, exits,
    size=taille_actions, size_type="amount",
    ts_stop=ts_stop, tp_stop=tp_stop,
    init_cash=100_000, fees=0.001, freq="1D",
    group_by=True, cash_sharing=True,    # un portefeuille partagé entre les 3 actifs
)

print(pf.stats())
print(f"Valeur finale : {pf.final_value():.0f} $")

```

À retenir : cet exemple combine filtre de régime, multi-indicateurs (SMA, MACD, RSI, ATR), dimensionnement par le risque (volatilité) et stops ATR dynamiques — le tout **vectorisé** sur les trois actifs simultanément. La fonction aligner règle un piège classique : les indicateurs vbt ajoutent un niveau de paramètre aux colonnes, qu'il faut réaligner avant toute algèbre booléenne. Deux limites du paradigme vectorisé apparaissent ici : le dimensionnement repose sur le **capital initial** (et non sur la valeur courante, dynamique) et le **plafonnement du nombre de positions** simultanées n'est pas direct — il faudrait un callback signal_func (Vectorbt Pro). La version LEAN du document jumeau gère ces deux points nativement.

Variante Pro : remplacer vbt.Portfolio par vbt.PF ; pour la limite de positions et un sizing dynamique, utiliser from_signals(..., signal_func_nb=...) ou from_order_func.

15. Conclusion

Vectorbt propose un cadre **vectorisé et ultra-rapide** : chaque exemple ci-dessus calcule les signaux sur toute la série d'un coup, puis simule le portefeuille sans boucle Python. Cette approche brille pour l'**exploration massive** de paramètres (Exemple 10) et l'analyse statistique (Exemple 9), au prix d'une modélisation un peu moins « marché réel » qu'un moteur événementiel — c'est à vous de veiller à l'absence de look-ahead bias.

Récapitulatif des réflexes acquis :

- Penser en trois temps : indicateurs → signaux booléens (entries, exits) → `Portfolio.from_signals`.
- Exploiter les classes d'indicateurs (`vbt.MA`, `vbt.RSI`, `vbt.BBANDS`, `vbt.MACD`) et leurs helpers de comparaison.
- Profiter des arguments intégrés : `size`, `size_type`, `sl_stop`, `tp_stop`, `fees`, `freq`.
- Tester des grilles entières en une passe, lire `pf.stats()`, et se méfier du sur-optimisation grâce à la validation hors-échantillon.

Le document jumeau montre comment **les mêmes onze stratégies** s'écrivent de façon **événementielle** avec QuantConnect LEAN — une comparaison très instructive entre les deux philosophies de backtesting.